COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/5/2026	**DATE SUBMITTED:**	8/10/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Sample code for Queue STL in C++:(sample.cpp)

sample.cpp

ILOA > sample.cpp > ...

```cpp
// C++ code to illustrate Queue in the Standard Template Library (STL)

#include <iostream>
#include <queue>

void display(std::queue<int> q)
{
    std::queue<int> c = q;

    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }

    std::cout << "\n";
}

int main()
{
    std::queue<int> a;

    // Insert elements into the queue
    a.push(10);
    a.push(20);
    a.push(30);

    std::cout << "The queue a is :";
    display(a);

    std::cout << "a.empty() : " << a.empty() << "\n";
    std::cout << "a.size() : " << a.size() << "\n";
    std::cout << "a.front() : " << a.front() << "\n";
    std::cout << "a.back() : " << a.back() << "\n";

    std::cout << "After a.pop() : ";
    a.pop();
    display(a);

    a.push(40);

    std::cout << "The queue a is : ";
    display(a);

    return 0;
}

```

Output:

```
The queue a is : 10 20 30
a.empty() : 0
a.size() : 3
a.front() : 10
a.back() : 30
After a.pop() :  20 30
The queue a is :  20 30 40
PS C:\Users\Justine\Desktop\Queue>
```

Observation:

After reading the instructions from the first part of the manual, queues use the FIFO (First-In, First-Out) principle, where enqueueing is done at the rear end using push(), and dequeuing occurs at the front end using pop(). Operations like a.push() enqueued the objects 10, 20, and 30 at the rear end of the queue. The condition !empty() in display() function allowed the loop to continue running until there were no objects left in the queue. In each iteration of the loop, the object at the front end was displayed using front() and then removed using pop(). Apart from front() and back(), size() and empty() were other methods which provided information about the status of the queue. The a.pop() operation removed the first object 10 from the queue, leaving behind 20 and 30 in the queue. Then a.push(40) inserted the object 40 at the rear end of the queue making the objects of the queue as 20, 30, and 40.

COMPUTER ENGINEERING

QUEZON CITY

Task: Create C++ code using Queue STL that will pass an array of students' names to the queue. Make observations on the operations performed and show screenshots of code and output console per operation.

queueArr.cpp

queueArr.cpp

ILOA > queueArr.cpp > ...

```cpp
#include <iostream>
#include <queue>
#include <string>

const int MaxSize = 5; // Maximum size of the queue

void display(std::queue<std::string> q)
{
    int i = 1;

    while (!q.empty())
    {
        std::cout << i << ": " << q.front() << "\n"; // Display the front element of the queue
        q.pop(); // Immediately removes the front element from the queue
        i++; // Then increments the counter for the next element in the array
    }

    std::cout << std::endl;
}

int main()
{
    //Array of students
    std::string studentName[MaxSize] = {
        "Ralph Braganza",
        "John Clarence",
        "Rojan Ambas",
        "James Verano",
        "Daniel Antivo"
    };

    std::queue<std::string> students;

    // Pass each name from the array into the queue
    for (int i = 0; i < MaxSize; i++)
    {
        students.push(studentName[i]);
    }

    // Display the students in the queue
    std::cout << "Students in the queue: \n";
    display(students);

    return 0;
}
```

Output:

```
Students in the queue:
1: Ralph Braganza
2: John Clarence
3: Rojan Ambas
4: James Verano
5: Daniel Antivo

PS C:\Users\Justine\Desktop\Queue>
```

Observation:

As compared to the sample code, the queue is now holding std::string type of elements rather than int elements, since the queue now has students' names. This is because the code now has an array containing five student names and MaxSize has been set to 5 to control the size of the array. There has also been an addition of a for loop to copy all the names of students from the array to the queue through students.push(studentName[i]).

B.1. Linked List Implementation

Tasks:

- **Operations and member functions described in this section must be in your developed code.**
- **Test the functions and show the output.**

mainB1.cpp

mainB1.cpp

ILOB1 > mainB1.cpp > ...

```cpp
#include <iostream>
#include "queueLL.h"
#include <queue>

int main() {
    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    std::cout << "Enqueueing:" << std::endl;
    enqueue(1, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl;

    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl;

    std::cout << "Queue:" << std::endl;
    display(front);
    std::cout << std::endl;

    std::cout << "Dequeueing:" << std::endl;
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl;

    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl;

    dequeue(&front, &back);
    dequeue(&front, &back); // Queue is empty

    return 0;
}
```

queueLL.h

queueLL.h

ILOB1 > queueLL.h > ...

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H
#include <iostream>

template <typename T>
class qNode {
    public:
        T data;
        qNode<T>* next;
};

template <typename T>
qNode<T> *new_Node(T newData) {
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>**backPtr) {
    //create a new node:
    qNode<T> *newNode = new_Node(newData);
```

```
24
25
26      // how do we know that the queue is empty? if the front and back pointers are both null, then the queue is empty
27      if(*frontPtr == nullptr && (*backPtr) == nullptr){
28          *frontPtr = newNode;
29          *backPtr = newNode;
30          return;
31      }
32
33      //point the back next to the newNode:
34      (*backPtr)->next = newNode;
35      //the newnode will be the new backPtr:
36      (*backPtr) = newNode;
37      return;
38  }
39
40  template <typename T>
41  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
42      //check if the queue is empty:
43      if(*frontPtr == nullptr && *backPtr == nullptr){
44          std::cout << "Queue is empty" << std::endl;
45          return;
46      }
47
48      //store the front node in a temporary variable:
49      qNode<T>* tempPtr = (*frontPtr);
50
51      if ((*frontPtr) == (*backPtr)) {
52          (*frontPtr) = nullptr;
53          (*backPtr) = nullptr;
54          delete tempPtr;
55          return;
56      }
57
58      //move the front pointer to the next node:
59      (*frontPtr) = (*frontPtr)->next;
60
61      tempPtr->next = nullptr;
62
63      //delete the temporary variable:
64      delete tempPtr;
65  }
66
67  template <typename T>
68  void display(qNode<T>* frontPtr) {
```

```
69      if (frontPtr == nullptr) {
70          std::cout << "Queue is empty" << std::endl;
71          return;
72      }
73
74      qNode<T>* current = frontPtr;
75      while (current != nullptr) {
76          std::cout << current->data << " ";
77          current = current->next;
78      }
79      std::cout << std::endl;
80  }
81
82  #endif
```

Output:

```
Enqueueing:
front: 1 back: 1
front: 1 back: 3
Queue:
1 2 3

Dequeueing:
front: 2 back: 3
front: 3 back: 3
Queue is empty
PS C:\Users\Justine\Desktop\Queue>
```

Analysis	
new_Node()	 This function creates a new node dynamically using new. It stores the given data in the node and sets its next pointer to nullptr because the new node does not point to another node yet. The function then returns the address of the newly created node.
enqueue()	 The enqueue() function will add an element to the rear of the queue. This is done by first creating a node through the use of new_Node(). If the front and back are both nullptr, it shows that the queue is empty. Therefore, both will be assigned the new node. In case there is content within the queue, the current back will point to the new node, and then the back will be assigned the new node.

dequeue()

```cpp
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    //check if the queue is empty:
    if(*frontPtr == nullptr && *backPtr == nullptr){
        std::cout << "Queue is empty" << std::endl;
        return;
    }

    //store the front node in a temporary variable:
    qNode<T>* tempPtr = (*frontPtr);

    if ((*frontPtr) == (*backPtr)) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete tempPtr;
        return;
    }

    //move the front pointer to the next node:
    (*frontPtr) = (*frontPtr)->next;

    tempPtr->next = nullptr;

    //delete the temporary variable:
    delete tempPtr;
}
```

The dequeue() function removes the element from the front of the queue. It first checks if the queue is empty. If the queue has only one node, both front and back are changed to nullptr, and the node is deleted. If there are multiple nodes, front moves to the next node, while the previous front node is deleted using delete.

display()

```cpp
template <typename T>
void display(qNode<T>* frontPtr) {
    if (frontPtr == nullptr) {
        std::cout << "Queue is empty" << std::endl;
        return;
    }

    qNode<T>* current = frontPtr;
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}
```

display() function shows all the items in the queue from the front end. This will be achieved by creating a temporary pointer variable called current that moves from one node to another with the help of the next pointer till current becomes nullptr.

Contents of main()

```
std::cout << "Enqueueing:" << std::endl;
enqueue(1, &front, &back);
std::cout << "front: " << front->data << " back: " << back->data << std::endl;
```

The first node is created. Since the queue is empty, both front and back point to 1.
front → 1 ← back

```
enqueue(2, &front, &back);
enqueue(3, &front, &back);
std::cout << "front: " << front->data << " back: " << back->data << std::endl;
```

These nodes are added to the back: **front → 1 → 2 → 3 ← back**

```
 std::cout << "Queue:" << std::endl;
 display(front);
 std::cout << std::endl;
```

Displays the queue: **1 2 3**

```
std::cout << "Dequeueing:" << std::endl;
dequeue(&front, &back);
std::cout << "front: " << front->data << " back: " << back->data << std::endl;

dequeue(&front, &back);
std::cout << "front: " << front->data << " back: " << back->data << std::endl;
```

1 is removed: **front → 2 → 3 ← back**
Then another dequeue removes 2: **front → 3 ← back**

```
dequeue(&front, &back);
dequeue(&front, &back); // Queue is empty
```

The third dequeue removes 3, leaving:
front → nullptr
back → nullptr

It is then called again but on an empty queue, so the program outputs:
Queue is empty

B.2. Array-Based Implementation

Tasks:

- **Operations and member functions described in this section must be in your developed code.**
- **Test the functions and show the output.**

mainB2.cpp

mainB2.cpp

ILOB2 > mainB2.cpp > ...

```cpp
#include <iostream>
#include "queueArr.h"
#include <queue>

int main() {
queueArr<int> q(6);

q.Enqueue(1);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(2);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(3);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(4);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(5);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(6);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(7);
q.Dequeue();
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Dequeue();
q.Dequeue();
q.Dequeue();
q.Dequeue();
q.Dequeue();
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

return 0;
}
```

queueLL.h

queueArr.h

ILOB2 > queueArr.h > queueArr<T> > Enqueue(T)

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>


template <typename T>
class queueArr {
    private:
        //pointer to dynamically allocated array
        T*  q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front, q_back;
```

```
14
15      public:
16          //constructor
17          queueArr(size_t q_capacity);
18
19          bool Empty();
20          size_t Size();
21          void Clear();
22
23          T Front();
24          T Back();
25
26          void Enqueue(T value);
27          void Dequeue();
28
29          //copy constructor:
30          queueArr(const queueArr& other);
31
32          //copy assignment operator
33          queueArr& operator = (const queueArr& other);
34
35          //destructor
36          ~queueArr();
37  };
38
39  template <typename T>
40  queueArr<T>::queueArr(size_t capacity) {
41      q_capacity = capacity;
42      q_size = 0;
43      q_front = 0;
44      q_back = -1;
45      q_array = new T[q_capacity];
46  }
47
48  template <typename T>
49  bool queueArr<T>::Empty(){
50      return q_size == 0;
51  }
52
53  template <typename T>
54  size_t queueArr<T>::Size(){
55      return q_size;
56  }
57
58  template <typename T>
59  void queueArr<T>::Clear(){
60      q_size = 0;
61      q_front = 0;
62      q_back = -1;
63  }
64
65  template <typename T>
66  T queueArr<T>::Front(){
67
68      if(Empty()){
69          std::cout << "The Queue is Empty ";
70          return T{};
71      }
72
73      return q_array[q_front];
74  }
75
76  template <typename T>
77  T queueArr<T>::Back(){
78
79      if(Empty()){
80          std::cout << "The Queue is Empty ";
81          return T{};
82      }
83
84      return q_array[q_back];
85  }
86
87  template <typename T>
88  void queueArr<T>::Enqueue(T value){
89      //check if the queue is full
90      if(q_size == q_capacity){
91          std::cout << "The Queue is Full\n";
92          return;
93      }
94      //compute for the q_back for circular (use the modulo operator)
95      q_back = (q_back + 1) % q_capacity;
```

```
96      // assign the q_array with the updated q_back to the value
97      q_array[q_back] = value;
98      //increment the q_size
99      q_size++;
100 }
101
102 template <typename T>
103 void queueArr<T>::Dequeue(){
104     //check if the queue is empty
105     if(Empty()){
106         std::cout << "The Queue is Empty ";
107         return;
108     }
109     //compute for the q_front for circular (use the modulo operator)
110     q_front = (q_front + 1) % q_capacity;
111     //decrement the q_size
112     q_size--;
113     // reset index when queue becomes empty
114     if(Empty()){
115         Clear();
116     }
117 }
118
119 template <typename T>
120 queueArr<T>::~queueArr(){
121     delete[] q_array;
122 }
123
124 #endif
```

Output

```
Front: 1 Back: 1 Size: 1
Front: 1 Back: 2 Size: 2
Front: 1 Back: 3 Size: 3
Front: 1 Back: 4 Size: 4
Front: 1 Back: 5 Size: 5
Front: 1 Back: 6 Size: 6
The Queue is Full
Front: 2 Back: 6 Size: 5
Front: The Queue is Empty 0 Back: The Queue is Empty 0 Size: 0
PS C:\Users\Justine\Desktop\Queue>
```

Analysis

Constructor

```
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}
```

The constructor creates the queue with a capacity of 6. At the beginning, q_size is 0, meaning the queue is empty. q_front is set to 0, while q_back is set to -1 because no elements have been added yet. The new keyword dynamically allocates an array that can hold 6 elements.

Empty()

```
template <typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}
```

Empty() determines whether there are any elements in the current queue. If the size of the queue (q_size) is 0, it will return true; otherwise, it will return false if there are elements in the queue. This function is also used by Front(), Back(), and Dequeue().

Size()

```
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}
```

The Size() function returns the current number of elements in the queue. Every successful Enqueue() increases q_size by 1, while every successful Dequeue() decreases it by 1.

Clear()

```
template <typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}
```

The Clear() function resets the queue back to its initial state. However, it does not delete the dynamically allocated array. Instead, it resets the size, front position, and back position so the queue can be used again from the beginning.

Front()

```
template <typename T>
T queueArr<T>::Front(){

    if(Empty()){
        std::cout << "The Queue is Empty ";
        return T{};
    }

    return q_array[q_front];
}
```

The Front() function returns the element currently at the front of the queue without removing it. It first checks if the queue is empty. If it is not empty, it accesses the array using q_front.

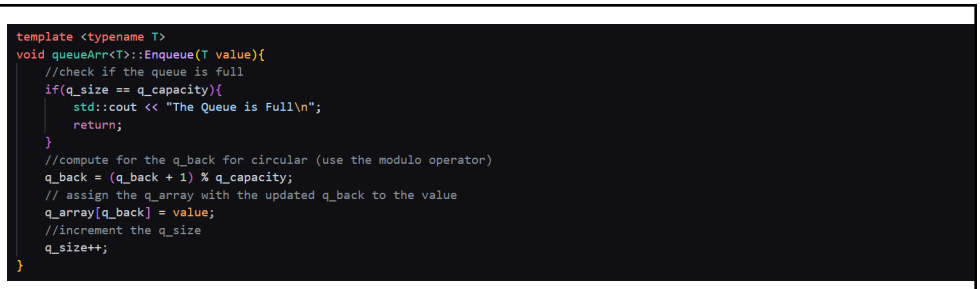
Back()

```
template <typename T>
T queueArr<T>::Back(){

    if(Empty()){
        std::cout << "The Queue is Empty ";
        return T{};
    }

    return q_array[q_back];
}
```

The Back() function returns the element currently at the back of the queue without removing it. It also checks whether the queue is empty before accessing the array.

Enqueue()	 The function Enqueue() will insert an element in the end of the queue. Initially, it determines whether the queue is full or not. Once the queue is found to have empty slots, q_back gets updated with the help of modulo operator and the new element is inserted at that index.
Dequeue()	 Dequeue() function removes the element at the front of the queue. Instead of physically deleting the element from the array, it moves q_front to the next position and decreases q_size. When the queue becomes empty, Clear() resets the queue's indexes.
Destructor	 The destructor function is called automatically whenever there is no need anymore for the queueArr object, at the end of the program execution. Because the constructor allocates memory by using new T[q_capacity], the destructor deallocates memory by using delete[] q_array. In this way, there will be no leaked memory after the queue is destroyed.
Contents of main()	 This creates a queue with a capacity of 6.

```cpp
template <typename T>
void queueArr<T>::Enqueue(T value){
    //check if the queue is full
    if(q_size == q_capacity){
        std::cout << "The Queue is Full\n";
        return;
    }
    //compute for the q_back for circular (use the modulo operator)
    q_back = (q_back + 1) % q_capacity;
    // assign the q_array with the updated q_back to the value
    q_array[q_back] = value;
    //increment the q_size
    q_size++;
}
```

```cpp
template <typename T>
void queueArr<T>::Dequeue(){
    //check if the queue is empty
    if(Empty()){
        std::cout << "The Queue is Empty ";
        return;
    }
    //compute for the q_front for circular (use the modulo operator)
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    // reset index when queue becomes empty
    if(Empty()){
        Clear();
    }
}
```

```cpp
template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}
```

```cpp
int main() {
queueArr<int> q(6);
```

```
q.Enqueue(1);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(2);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(3);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(4);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(5);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;

q.Enqueue(6);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
```

Enqueues 1-6 where the current queue becomes:
Front → 1 2 3 4 5 6 ← Back

```
q.Enqueue(7);
q.Dequeue();
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
```

Enqueues 7 but the queue is already full so it doesn't add it, then removes enqueue 1 logically by moving the front forward making the new size from 6 to 5:
Front → 2 3 4 5 6 ← Back

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

mainC.cpp

mainC.cpp ●

ILOC > mainC.cpp > main()

```cpp
#include <iostream>
#include <limits>
#include "printJob.h"

int main()
{
Printer printer;
int choice;

    do
    {
        // Menu display
        std::cout << "\n===== | SHARED PRINTER | =====\n";
        std::cout << "1. Add Print Job\n";
        std::cout << "2. Remove Print Job\n";
        std::cout << "3. Print Next Job\n";
        std::cout << "4. Print All Jobs\n";
        std::cout << "5. Display Queue\n";
        std::cout << "6. Exit\n";
        std::cout << "\n============================\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        // Error checking for menu input
        if (std::cin.fail()){
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid input. Please enter a number.\n";
            continue;
        }

        if (choice == 1){
            int id;
            std::string name;
            int pages;

            std::cout << "Enter Job ID: ";
            std::cin >> id;

            if (std::cin.fail()){
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                std::cout << "Invalid Job ID.\n";
                continue;
            }

            std::cout << "Enter User Name: ";
            std::cin >> name;
            std::cout << "Enter Number of Pages: ";
            std::cin >> pages;

            if (std::cin.fail()){
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                std::cout << "Invalid number of pages.\n";
                continue;
            }

            printer.addJob(Job(id, name, pages));

        }else if (choice == 2){
            printer.removeJob(); // Remove the next job in the queue
        }else if (choice == 3){
            printer.printNext(); // Print the next job in the queue
        }else if (choice == 4){
            printer.printAll(); // Print all jobs in the queue
        }else if (choice == 5){
            printer.displayQueue(); // Display the current queue of jobs
        }else if (choice == 6){
            std::cout << "Exiting program...\n"; // Exit the program
        }else {
            std::cout << "Invalid choice. Please choose from 1 to 6.\n"; // Invalid choice
        }

    } while (choice != 6);

return 0;
}
```

printJob.h

printJob.h

ILOC > printJob.h > Printer > printAll()

```
#ifndef PRINTJOB_H
#define PRINTJOB_H
#include <iostream>
#include <string>

class Job {
    public:
    int id;
    std::string name;
    int pages;
        Job(){
            id = 0;
            name = "";
            pages = 0;
        }

        Job(int jobId, std::string userName, int pageNum){
            id = jobId;
            name = userName;
            pages = pageNum;
        }
    };

class Printer {
    private:
    static const int maxJobs = 5;

        Job jobs[maxJobs];
        int front = 0;
        int back = -1;
        int size = 0;

    public:
        bool isEmpty(){
            return size == 0;
        }

        bool Full(){
            return size == maxJobs;
        }

        void addJob(Job job){
            if (Full()){
                std::cout << "======= |      QUEUE IS FULL      | =======\n";
                std::cout << "Cannot add another print job.\n";
                return;
            }
            back = (back + 1) % maxJobs;
            jobs[back] = job;
            size++;
            std::cout << "======= | PRINT JOB ADDED SUCCESSFULLY | =======\n";
        }

        void removeJob(){
            if (isEmpty()){
                std::cout << "======= |     QUEUE IS EMPTY      | =======\n";
                std::cout << "There are no jobs to remove.\n";
                return;
            }
            std::cout << "======= |    PRINT JOB REMOVED     | =======\n";
            std::cout << "Job " << jobs[front].id << " from "
                      << jobs[front].name << " has been removed.\n";
            front = (front + 1) % maxJobs;
            size--;

            if (isEmpty()){
                front = 0;
                back = -1;
            }
        }

        void printNext(){
            if (isEmpty()){
                std::cout << "======= |     QUEUE IS EMPTY      | =======\n";
                std::cout << "There are no jobs to print.\n";
                return;
            }

```

```
            std::cout << "============ |    PRINTING    | ============\n";
            std::cout << "Job ID: " << jobs[front].id << "\n";
            std::cout << "User:   " << jobs[front].name << "\n";
            std::cout << "Pages:  " << jobs[front].pages << "\n";
            std::cout << "======= |     PRINT JOB COMPLETED     | =======\n";

            front = (front + 1) % maxJobs;
            size--;

            if (isEmpty()){
                front = 0;
                back = -1;
            }
        }

        void printAll(){
            if (isEmpty()){
                std::cout << "======= |     QUEUE IS EMPTY     | =======\n";
                std::cout << "There are no jobs to print.\n";
                return;
            }
            std::cout << "\n======= |     PRINTING ALL JOBS     | =======\n";

            while (!isEmpty()){
                printNext();
            }
            std::cout << "======== |    ALL JOBS PRINTED    | ========\n";
        }

        void displayQueue(){
            if (isEmpty()){
                std::cout << "======= |     QUEUE IS EMPTY     | =======\n";
                std::cout << "There are no jobs in the queue.\n";
                return;
            }
            std::cout << "\n========== |     CURRENT QUEUE     | ==========\n";
            int current = front;

            for (int i = 0; i < size; i++){
                std::cout << "Job ID: " << jobs[current].id << "\n";
                std::cout << "User:   " << jobs[current].name << "\n";
                std::cout << "Pages:  " << jobs[current].pages << "\n";

                if (i < size - 1){
                    std::cout << "-----------------------------------------\n";
                }
                current = (current + 1) % maxJobs;
            }
            std::cout << "=========================================\n";
        }

    };

#endif
```

Outputs:

Menu:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

================================
Enter choice:
```

Add Print Job:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

================================
Enter choice: 1
Enter Job ID: 1437
Enter User Name: Ralph
Enter Number of Pages: 21
======= | PRINT JOB ADDED SUCCESSFULLY | =======
```

Queue is Full:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

================================
Enter choice: 1
Enter Job ID: 3424
Enter User Name: Almirol
Enter Number of Pages: 43
=======|      QUEUE IS FULL      |=======
Cannot add another print job.
```

Remove Print Job:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

================================
Enter choice: 2
=======|    PRINT JOB REMOVED    |=======
Job 1437 from Ralph has been removed.
```

Queue is Empty:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

==============================
Enter choice: 2
======= |     QUEUE IS EMPTY     | =======
There are no jobs to remove.
```

Print Next Job:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

==============================
Enter choice: 3
============ |    PRINTING    | ============
Job ID: 1437
User:   Ralph
Pages:  6
======= |     PRINT JOB COMPLETED     | =======
```

Print All Jobs:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

==============================
Enter choice: 4

======= |     PRINTING ALL JOBS     | =======
============ |    PRINTING    | ============
Job ID: 2005
User:   James
Pages:  7
======= |     PRINT JOB COMPLETED     | =======
============ |    PRINTING    | ============
Job ID: 1620
User:   Uchi
Pages:  21
======= |     PRINT JOB COMPLETED     | =======
============ |    PRINTING    | ============
Job ID: 2443
User:   Arensol
Pages:  3
======= |     PRINT JOB COMPLETED     | =======
============ |    PRINTING    | ============
Job ID: 4378
User:   Bal
Pages:  1
======= |     PRINT JOB COMPLETED     | =======
======== |    ALL JOBS PRINTED    | ========
```

Display Queue

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

==============================
Enter choice: 5

========== |     CURRENT QUEUE     | ==========
Job ID: 2005
User:   James
Pages:  7
---------------------------------------------
Job ID: 1620
User:   Uchi
Pages:  21
---------------------------------------------
Job ID: 2443
User:   Arensol
Pages:  3
---------------------------------------------
Job ID: 4378
User:   Bal
Pages:  1
=============================================
```

Display Queue (Empty):

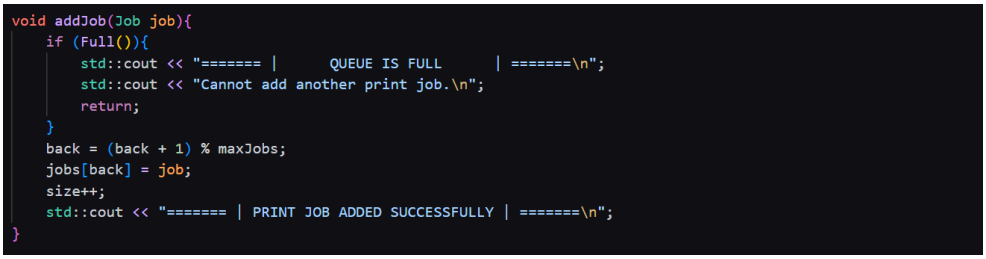
```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

==============================
Enter choice: 5
======= |     QUEUE IS EMPTY     | =======
There are no jobs in the queue.
```

Exiting Program:

```
===== | SHARED PRINTER | =====
1. Add Print Job
2. Remove Print Job
3. Print Next Job
4. Print All Jobs
5. Display Queue
6. Exit

==============================
Enter choice: 6
Exiting program...
```

Analysis	
Job()	 The constructor sets up a Job object using default values. The ID and page numbers are both 0, and the name is an empty string. This provides default values for the object before any print job is actually created.  This constructor helps create a new print job using the job ID, name of the user, and the number of pages. All these parameters can then be stored within the Job object. This helps in passing all the parameters of one print job in the printer queue.
isEmpty()	 isEmpty() function determines if there are any jobs assigned in the queue. If the size is found to be zero, then it results in the output true; otherwise, it results in the output false. Functions such as removeJob(), printNext(), and displayQueue() use the function isEmpty().
Full()	 Full() checks to see if the queue has hit its maximum capacity of 5 jobs (this is what I set it to). This is used by addJob() to make sure no more jobs can be added to the array once it's full.
addJob()	 The addJob() function (this is the enqueuing function) is used to add a new print job at the back of the queue. Initially, the addJob () function checks through the Full() function to see whether the queue is full or not. If it is not full, then the new print job is added.

removeJob()

```
void removeJob(){
    if (isEmpty()){
        std::cout << "======= |     QUEUE IS EMPTY     | =======\n";
        std::cout << "There are no jobs to remove.\n";
        return;
    }
    std::cout << "======= |    PRINT JOB REMOVED    | =======\n";
    std::cout << "Job " << jobs[front].id << " from "
              << jobs[front].name << " has been removed.\n";
    front = (front + 1) % maxJobs;
    size--;

    if (isEmpty()){
        front = 0;
        back = -1;
    }
}
```

The removeJob() function (this is the dequeuing function) removes the job from the head of the queue. First, it checks whether the queue is empty or not. If the queue is not empty, front is incremented by one, and size is decremented by one.

printNext()

```
void printNext(){
    if (isEmpty()){
        std::cout << "======= |     QUEUE IS EMPTY     | =======\n";
        std::cout << "There are no jobs to print.\n";
        return;
    }

    std::cout << "============ |    PRINTING    | ============\n";
    std::cout << "Job ID: " << jobs[front].id << "\n";
    std::cout << "User:   " << jobs[front].name << "\n";
    std::cout << "Pages:  " << jobs[front].pages << "\n";
    std::cout << "======= |     PRINT JOB COMPLETED     | =======\n";

    front = (front + 1) % maxJobs;
    size--;

    if (isEmpty()){
        front = 0;
        back = -1;
    }
}
```

The printNext() function processes the first job in the queue. It displays the job's ID, user, and number of pages before moving the front position forward. The size is also decreased because the job has already been printed and is no longer pending.

printAll()

```
void printAll(){
    if (isEmpty()){
        std::cout << "======= |     QUEUE IS EMPTY     | =======\n";
        std::cout << "There are no jobs to print.\n";
        return;
    }
    std::cout << "\n======= |     PRINTING ALL JOBS     | =======\n";

    while (!isEmpty()){
        printNext();
    }
    std::cout << "======== |    ALL JOBS PRINTED    | ========\n";
}
```

The printAll() function processes every pending job in the queue. It uses while (!isEmpty()) to continue calling printNext() until there are no jobs left.

displayQueue()

```
void displayQueue(){
    if (isEmpty()){
        std::cout << "======= |     QUEUE IS EMPTY     | =======\n";
        std::cout << "There are no jobs in the queue.\n";
        return;
    }
    std::cout << "\n========== |     CURRENT QUEUE     | ==========\n";
    int current = front;

    for (int i = 0; i < size; i++){
        std::cout << "Job ID: " << jobs[current].id << "\n";
        std::cout << "User:   " << jobs[current].name << "\n";
        std::cout << "Pages:  " << jobs[current].pages << "\n";

        if (i < size - 1){
            std::cout << "----------------------------------------\n";
        }
        current = (current + 1) % maxJobs;
    }
    std::cout << "========================================\n";
}
```

The displayQueue() method displays all the pending jobs in the queue without dequeuing them. Temporary variable current is declared to ensure that the real front index is not changed. The iteration begins from the front and continues for all the jobs till the end which is determined by the current size of the queue.

Why did I use an array implementation instead of a linked list implementation?

To put it simply, I feel more at ease implementing the array since this is one of the data structures that I have implemented within my first year in college. In this particular activity, the maximum number of jobs in the printer queue is also limited to 5 and this makes the use of array a logical decision, since in advance I know the maximum capacity of space available for the queue. The use of array enabled me to concentrate on comprehending the queue data structure and operations like front, back, and size used to keep the FIFO principle. Moreover, I used the modulo operator in order to create the circular array that would enable reuse of previous elements of the array. Compared to the linked list implementation, I am not that proficient in dealing with pointers, dynamic memory allocation, and deletion of nodes.

Moreover, the deletion of data in an array becomes easy on my side since I don't have to delete nodes or deal with pointers as I would in a linked list. In this activity, the removal of a job is just a matter of changing the position of the front and reducing the size, thus making the job deleted from the queue. Although the data will still be present in that particular array position temporarily, the place can then be used again by overwriting with new jobs.

Conclusion:

Through the activity, I was able to learn how to implement a queue without utilizing STL through an array implementation using the FIFO principle. Much of the code had been derived from ILO A, ILO B1, and ILO B2 specifically on arrays, classes, and data handling in memory. I was able to learn how the front, back, and size function operated to maintain the queue. I learned how the modulo operator can be used to simulate a circular array to enable reusing of previous elements.

Additionally, the activity taught me how the queue works in practical terms. In ILOC, I programmed functions like addJob(), removeJob(), printNext(), printAll(), displayQueue(), isEmpty(), and Full(). While addJob() inserts jobs at the back position, removeJob() and printNext() processes the jobs at the front end. isEmpty() and Full() are also used to ensure invalid operations are prevented. I encountered many difficulties in programming the queue and had to search for solutions through the errors that I've gotten while coding, to learn where the error is located

and how to fix it. There were also input errors in my program making it enter into an invalid-input loop.

Through supplementary activity, I got an opportunity to use queue concepts learned during the previous ILOs in a simulated printer system. Unlike the simple storage of integers, in this case, I developed a Job object which included job id's, name of the user and number of pages. Then I developed a Printer object to handle these Jobs. It relied heavily on concepts learned through ILOA, ILOB1 and ILOB2, but ILO C came into play as I put together the functions that were taught to me while I was performing this activity. Through development of printNext() and printAll(), I got the practical understanding of how queues can be applied in real life (or at least in a printing job).

Overall, I believe that I did decently in this assignment, as I was able to implement a working queue and use it to simulate the printing simulation. I have faced several mistakes along the way and was using trial and error in order to comprehend how circular array, moving of front and back works, as well as handling invalid user inputs. Nevertheless, such problems were helping me to understand the code rather than just memorizing the structure of it. Moreover, I now understand that while implementing an array-based queue, it does not necessarily have to physically delete old data; instead, the queue can stop considering it part of the queue by updating front and size, and the space can later be reused. My biggest improvement should be related to gaining experience in implementation of data structures from scratch, particularly, linked lists and pointers, rather than being reliant on implementation of array. Additionally, I need to work on my efficiency in coding, as this assignment consumed approximately my entire afternoon and evening (it took me 8 hours of my life).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google Overview and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, code assistance and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to guide me on what I need to do, help debug and grammar. The code was heavily referenced from ILOA, ILOB and ILOB2 and modified by me to fit the criteria being asked. It was also used to fix alignment for the ILOC menu.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.